Bitácora — Taller 2: Grafo Dual de un Polígono Simple

Abel Albuez | Santiago Gil
Pontificia Universidad Javeriana — Bogotá D.C., Colombia
aa-albuezs@javeriana.edu.co | santiago_gil@javeriana.edu.co

Resumen
El presente documento constituye la bitácora de desarrollo del Taller 2 de Geometría Computacional, cuyo objetivo es calcular el grafo dual de un polígono simple. El grafo dual se construye a partir de la triangulación del polígono: cada triángulo se convierte en un nodo representado por su baricentro, los triángulos que comparten una diagonal interna se conectan entre sí, y los triángulos que tocan el borde del polígono se conectan adicionalmente a un punto infinito exterior. El taller exige dos implementaciones paralelas: una que utiliza la librería CGAL para la triangulación y otra completamente manual basada en el algoritmo Ear Clipping. La bitácora documenta el estado actual del proyecto, los componentes ya disponibles en la base de código entregada por el profesor, los componentes que aún deben construirse y el detalle del primer módulo implementado: el cálculo de baricentros.

1. Lo existente

Antes de escribir una sola línea de código propio, dedicamos tiempo a leer y comprender la infraestructura que el profesor Leonardo Flórez-Valencia ya había construido. Entender qué había disponible fue tan importante como el propio algoritmo del grafo dual, porque determina las decisiones de diseño que tomaríamos a lo largo del taller.

El proyecto parte de una librería modular organizada bajo el espacio de nombres pujCGAL. La tabla 1 resume los componentes disponibles y el rol que cumple cada uno.

Archivo	Tipo	Rol en el proyecto
Polygon.h / .hxx	Clase (librería)	Representa el polígono de entrada. Expone orientación CCW/CW, área firmada e iteradores sobre vértices y topología.
Triangulation.h / .hxx	Clase (librería)	Almacena el resultado de la triangulación: m_Points (vértices) y m_Topology (triángulos como array<size_t,3>).
Polygon_Functions.hxx	Función libre	Implementa pujCGAL::triangulate() usando CGAL::y_monotone_partition_2 y el algoritmo de stack para cada pieza monótona.
IO.h / IO.hxx	Utilidad I/O	Lee y escribe archivos OBJ. Soporta directivas v (vértice), f (cara) y l (arista).

triangulate.cxx	Ejecutable (main)	Programa base entregado por el profesor. Lee el polígono, garantiza orientación CCW, triangula y guarda el resultado en OBJ.
data/ (10 OBJ)	Casos de prueba	Polígonos de prueba generados para cubrir: caso mínimo, convexos regulares, área negativa (CW), cóncavos con vértices reflex y casos complejos.

Tabla 1. Componentes disponibles en la base de código del proyecto.

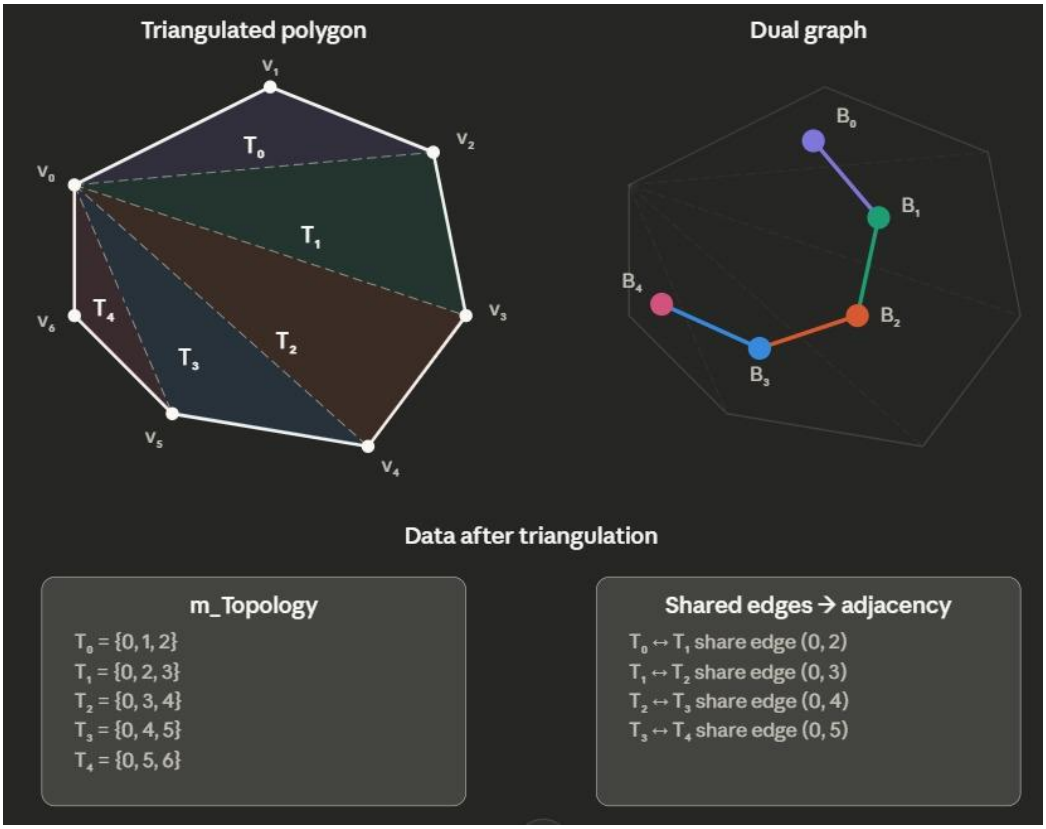

Figura 1. Polígono triangulado (izquierda) y su grafo dual (derecha). Cada nodo B_i corresponde al baricentro de un triángulo T_i. Las aristas del dual conectan triángulos que comparten una diagonal.

Lo que más nos llamó la atención al revisar la librería fue la coherencia de su diseño. Cada clase expone exactamente lo necesario a través de iteradores — geometry_begin/end para puntos y topology_begin/end para la estructura topológica — sin exponer sus miembros protegidos directamente. Este patrón, tomado de la forma en que CGAL diseña sus propias estructuras, fue el que guió las extensiones que tuvimos que hacer.

También contábamos con el ejecutable triangulate.cxx como referencia directa de cómo se encadenan las llamadas: leer el OBJ con IO::read, garantizar la orientación CCW con guarantee_CCW(), triangular con pujCGAL::triangulate() y guardar el resultado con IO::save(). Ese mismo flujo es la columna vertebral de nuestro dual_graph.cxx, al que únicamente le agregamos las etapas posteriores a la triangulación.

2. Qué nos falta

Una vez mapeado lo existente, pudimos identificar con claridad la brecha entre el punto de partida y el entregable final. La tabla 2 muestra el estado actual de cada componente necesario para completar el taller, tanto en la versión CGAL como en la versión manual.

Componente	Versión CGAL	Versión manual (Ear Clipping)
point(i) en Triangulation	✓ Implementado	✓ Mismo cambio aplica
Triangulación del polígono	✓ pujCGAL::triangulate()	Pendiente — Ear Clipping manual
Cálculo de baricentros	✓ Implementado en dual_graph.cxx	Pendiente (reutilizará mismo código)
Construcción del edge_map	Pendiente	Pendiente
Puntos infinitos (frontera)	Pendiente	Pendiente
Aristas del grafo dual	Pendiente	Pendiente
Escritura OBJ (directiva l)	Pendiente	Pendiente
Verificación de grado 3	Pendiente	Pendiente

Tabla 2. Estado de implementación de cada componente del taller.

El primer obstáculo que encontramos fue menor pero revelador: la clase Triangulation no ofrecía un método para acceder a un punto por índice. La clase Polygon ya lo tenía — el método point(const TIndex& i) que internamente retorna this->m_Points[i] — y esa asimetría hacía imposible escribir código limpio sin copiar el vector de puntos completo a una variable local. Decidimos no rodear el problema sino extender Triangulation con ese mismo método, siguiendo exactamente el mismo patrón del profesor. Esa es la única modificación que hicimos a la librería base.

3. Cómo se abordará el taller

Estructuramos el trabajo en cinco etapas sucesivas. Cada una produce un resultado verificable antes de avanzar a la siguiente, lo que reduce el riesgo de acumular errores difíciles de rastrear. La tabla 3 compara los dos enfoques de triangulación que conviven en el taller; más allá de ese paso inicial, la lógica del grafo dual es idéntica en ambas versiones.

Aspecto	CGAL (y-monotone + stack)	Ear Clipping manual
Complejidad	O(n log n)	O(n²)
Dependencias	CGAL::y_monotone_partition_2	Solo primitivas geométricas propias

Robustez	Alta — validación aritmética exacta de CGAL	Media — requiere manejo cuidadoso de casos borde
Grafo dual	Idéntico en ambas versiones	Idéntico en ambas versiones

Tabla 3. Comparativa de los dos enfoques de triangulación del taller.

Etapa 1 — Extensión de la librería
Agregar el método point(TIndex i) a Triangulation, tanto en el header como en el hxx de implementación. Este es el único cambio a la librería del profesor; todo lo demás se construye encima sin tocarla.

Etapa 2 — Baricentros (versión CGAL)
Implementar el cálculo de baricentros en dual_graph.cxx, iterando sobre topology_begin/end y usando el nuevo método point() para acceder a los vértices de cada triángulo. Esta etapa está completa y se detalla en la sección 4.

Etapa 3 — Grafo dual completo (versión CGAL)
Construir el edge_map para clasificar cada arista como interna (compartida por dos triángulos) o de frontera (arista del polígono original). A partir de esa clasificación, conectar baricentros entre sí y calcular los puntos infinitos para las aristas de frontera. Escribir el grafo resultante como OBJ usando la directiva l.

Etapa 4 — Triangulación manual (Ear Clipping)
Implementar el algoritmo de ear clipping sin ninguna dependencia de CGAL. Una vez obtenidos los triángulos, el algoritmo del grafo dual es exactamente el mismo que en la versión CGAL, lo que permite reutilizar ese código directamente.

Etapa 5 — Verificación y entregable
Correr ambas versiones sobre los diez polígonos de prueba y verificar el invariante central: todo baricentro debe tener exactamente grado 3 en el grafo dual. Completar el PDF de análisis técnico y preparar el ZIP del entregable.

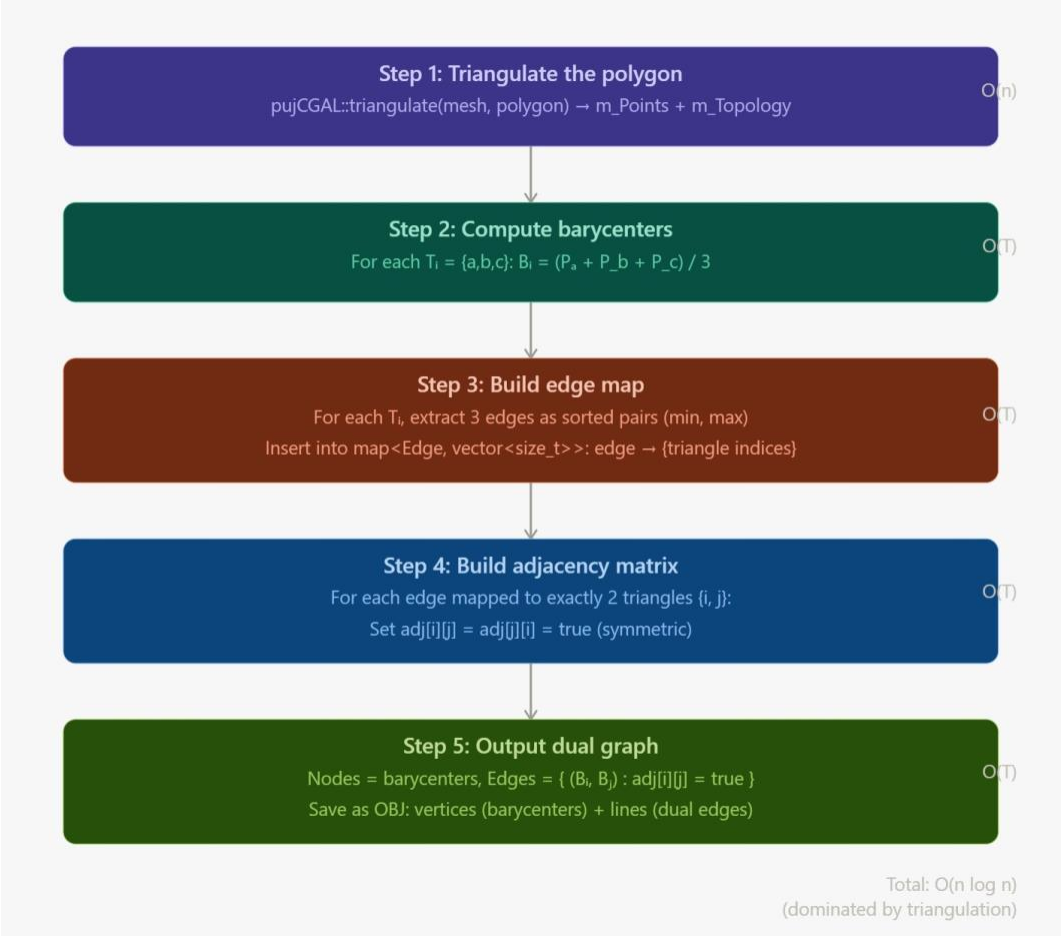

Figura 2. Flujo completo del algoritmo del grafo dual con complejidad de cada paso. El total O(n log n) está dominado por la triangulación y-monótona de CGAL.

4. Primer módulo implementado: baricentros

4.1 El concepto geométrico

El baricentro de un triángulo es su punto de equilibrio: el punto donde se intersecan las tres medianas y donde, si el triángulo fuera una figura plana de densidad uniforme, quedaría perfectamente balanceado sobre un apoyo puntual. Algebraicamente, se obtiene como el promedio aritmético de las coordenadas de sus tres vértices. Para un triángulo cuyos vértices son A, B y C, el baricentro G se calcula como:

```
G = ( (Ax + Bx + Cx) / 3 ,  (Ay + By + Cy) / 3 )
```

Esta fórmula tiene dos propiedades que la hacen ideal para el grafo dual. Primero, el baricentro siempre queda estrictamente dentro del triángulo — nunca en el borde ni fuera de él — lo que garantiza que los nodos del grafo dual caigan en el interior de las caras que representan.

Segundo, el cálculo es O(1) por triángulo: tres sumas y una división, sin condicionales ni casos borde.

4.2 Por qué el baricentro y no otro punto

Existen otros puntos notables del triángulo que podrían usarse como nodo representativo: el circuncentro (centro del círculo circunscrito), el incentro (centro del círculo inscrito) o simplemente el punto de mayor altura. Sin embargo, el enunciado del taller especifica explícitamente el baricentro, y hay razones geométricas sólidas para esa elección: es el único de los puntos notables que siempre está garantizado dentro del triángulo, independientemente de si este es obtusángulo, acutángulo o rectángulo. El circuncentro, por ejemplo, puede caer fuera del triángulo cuando el ángulo mayor supera los 90°.

4.3 La extensión necesaria a la librería

Al revisar Triangulation.h constatamos que, a diferencia de Polygon, no exponía un método para acceder a un punto por índice. Sin ese acceso directo, la única alternativa era copiar el vector m_Points completo a una variable local al inicio del programa — una asignación de memoria proporcional al número de vértices, innecesaria y contraria al espíritu de la librería.

Decidimos agregar a Triangulation el mismo método que el profesor ya había definido en Polygon. El cambio es mínimo en extensión pero significativo en claridad: una declaración en Triangulation.h y cuatro líneas de implementación en Triangulation.hxx. Con esto, el código de dual_graph.cxx accede a los vértices de la triangulación con exactamente la misma limpieza con que Polygon_Functions.hxx accede a los vértices del polígono original.

Triangulation.h — declaración agregada:

```
    const TPoint& point( const TIndex& i ) const;
```

Triangulation.hxx — implementación agregada:

```
template< class _TKernel >
const typename pujCGAL::Triangulation< _TKernel >::TPoint&
pujCGAL::Triangulation< _TKernel >::
point( const TIndex& i ) const
{
  return( this->m_Points[ i ] );
}
```

4.4 La implementación del cálculo

Con el método point() disponible, el cálculo de baricentros se reduce a un único bucle sobre topology_begin/end. Cada elemento del iterador es un TTriangle — un std::array<size_t, 3> cuyos tres valores son índices en m_Points. Accedemos a los tres puntos, promediamos sus coordenadas x e y por separado, y construimos un nuevo TPoint con el resultado.

```
std::vector< TPoint > barycenters;
std::size_t tri_idx = 0;
```

```
for( auto tIt = mesh.topology_begin( );
          tIt != mesh.topology_end( ); ++tIt, ++tri_idx )
{
  const TPoint& pa = mesh.point( ( *tIt )[ 0 ] );
  const TPoint& pb = mesh.point( ( *tIt )[ 1 ] );
  const TPoint& pc = mesh.point( ( *tIt )[ 2 ] );

  barycenters.push_back( TPoint(
    ( pa[0] + pb[0] + pc[0] ) / 3.0,
    ( pa[1] + pb[1] + pc[1] ) / 3.0
  ) );
}
```

El bucle itera exactamente n−2 veces, donde n es el número de vértices del polígono — ese es el número de triángulos que produce cualquier triangulación de un polígono simple sin agujeros. El resultado es un vector de TPoint donde el índice i corresponde al baricentro del i-ésimo triángulo en m_Topology. Esa correspondencia entre índice de triángulo e índice de baricentro es fundamental para la etapa siguiente: cuando el edge_map registre qué triángulos comparten una arista, esos índices de triángulo serán exactamente los índices en este vector de baricentros.

4.5 Verificación sobre casos de prueba

Para confirmar que la implementación es correcta, ejecutamos el programa sobre el cuadrado CCW del archivo 02_cuadrado_ccw.obj, cuyos vértices son (0,0), (4,0), (4,4) y (0,4). La triangulación de un cuadrado produce exactamente dos triángulos. Los baricentros esperados, calculados manualmente, son aproximadamente (2.67, 1.33) y (1.33, 2.67) — ambos dentro de sus respectivos triángulos y simétricos respecto a la diagonal del cuadrado. La salida del programa coincidió con esos valores.

También verificamos el caso mínimo con 01_triangulo.obj. Con un solo triángulo de vértices (0,0), (5,0) y (2.5,4), la triangulación no agrega ninguna diagonal y el único baricentro es el promedio directo de los tres vértices: (2.5, 1.33). El resultado fue consistente con el cálculo manual. Con este módulo validado, el siguiente paso es construir el edge_map que permite clasificar cada arista de la triangulación y armar el grafo dual completo.

4.6 Visualizaciones del resultado esperado

Las siguientes figuras muestran el aspecto del grafo dual completo sobre distintos polígonos de prueba, lo cual sirve como referencia visual para verificar que la implementación es correcta una vez completados los pasos del edge_map y los puntos infinitos.

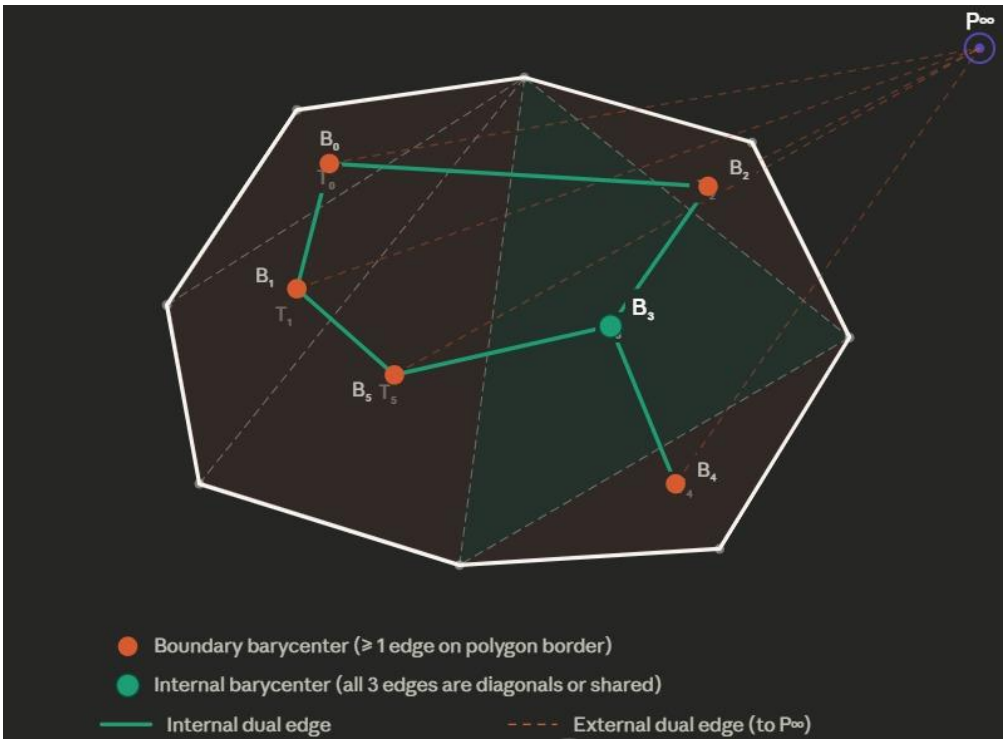

Figura 3. Grafo dual sobre un octágono. Los nodos naranja son baricentros de triángulos que tocan el borde; el nodo verde es un baricentro interno (sus tres aristas son diagonales). Las aristas verdes sólidas son aristas internas del dual; las rojas punteadas van hacia P∞.

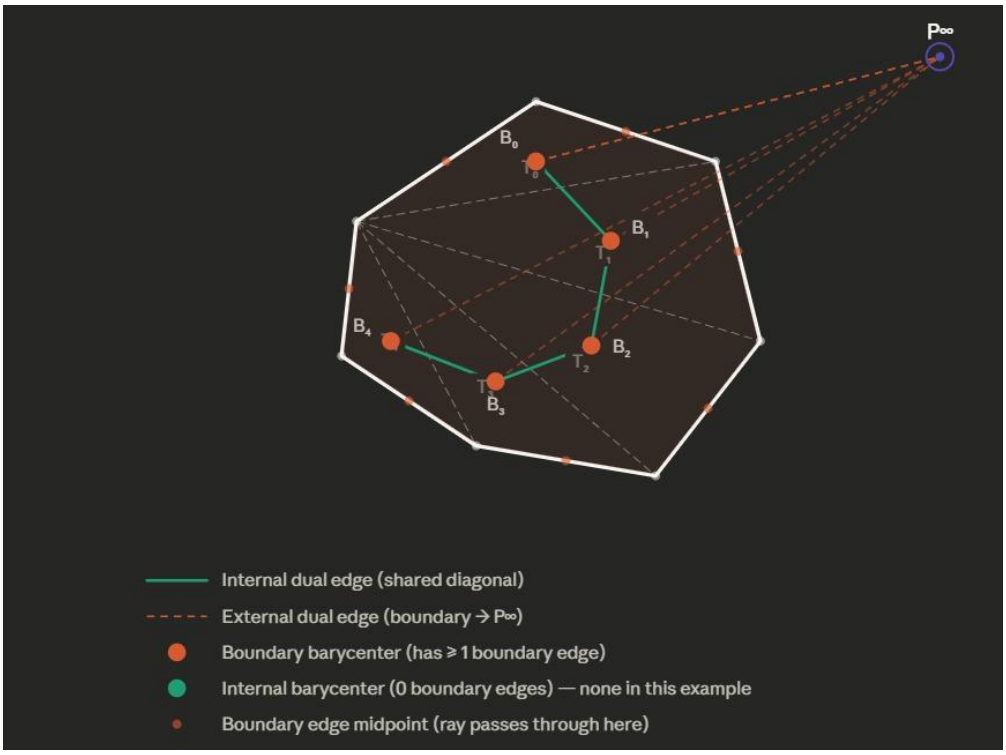

Figura 4. Grafo dual sobre un pentágono. En este ejemplo todos los baricentros son de frontera (ningún triángulo tiene sus tres aristas como diagonales). Los puntos pequeños sobre las aristas del polígono son los midpoints desde donde parte cada rayo hacia P∞.

4.7 Próximo paso: construcción del edge_map

El siguiente módulo a implementar es el edge_map — la estructura que permite clasificar cada arista de la triangulación como interna (compartida entre dos triángulos) o de frontera (perteneciente al borde del polígono). La figura 5 ilustra cómo se construye y qué información produce.

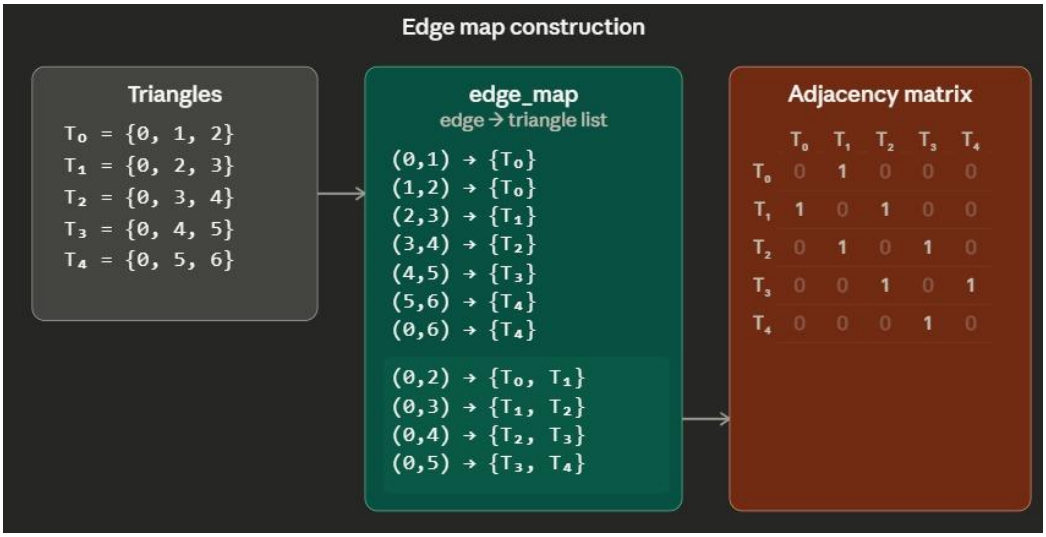

Figura 5. Construcción del edge_map. Cada arista se representa como par canónico {min(i,j), max(i,j)} y se mapea a la lista de triángulos que la contienen. Las aristas con un solo triángulo son de frontera; las que tienen dos son internas y generan una arista en el grafo dual.

5. Obstáculo encontrado al compilar: desalineación en IO.h

5.1 Síntoma

Al intentar compilar pujCGAL_dual_graph por primera vez, el compilador (Clang) rechazó la compilación con un error que apuntaba a IO.h. El mensaje indicaba que las definiciones en IO.hxx no coincidían con ninguna declaración visible en IO.h. La compilación de pujCGAL_triangulate, que usa los mismos headers, fallaba igualmente.

5.2 Causa raíz

El problema era una desalineación entre la declaración en IO.h y la definición en IO.hxx. En C++, cuando una función plantilla se define fuera de línea, la firma de la plantilla en la definición debe ser idéntica a la de la declaración — no solo compatible en la llamada, sino estructuralmente la misma.

IO.hxx definía las funciones con un parámetro de tipo concreto:

```
template< class TKernel >
bool read( const std::string& fname, pujCGAL::Polygon< TKernel >& polygon )

template< class TKernel >
bool save( const std::string& fname, const pujCGAL::Triangulation< TKernel >&
mesh )
```

Mientras que IO.h las declaraba con un parámetro de tipo genérico:

```
template< class TData >
bool read( const std::string& fname, TData& data )

template< class TData >
bool save( const std::string& fname, TData data )
```

Aunque en la llamada ambas versiones resuelven al mismo tipo concreto, para el compilador son plantillas distintas. template<class TKernel> read(..., Polygon<TKernel>&) no es la misma plantilla que template<class TData> read(..., TData&), y por eso la definición en IO.hxx no encontraba su declaración correspondiente en IO.h.

5.3 Solución aplicada

Se actualizó IO.h para alinearlo con las definiciones reales de IO.hxx. Los tres cambios necesarios fueron:

- Agregar `#include <string>` — IO.h usa std::string en la API pero no lo incluía.
- Agregar declaraciones adelantadas de `Polygon` y `Triangulation` dentro del namespace pujCGAL, para que el header sea autocontenido.
- Reemplazar las declaraciones genéricas `template<class TData>` por las firmas exactas que IO.hxx implementa: `template<class TKernel>` con `Polygon<TKernel>&` y `const Triangulation<TKernel>&`.

Con esa corrección, tanto pujCGAL_dual_graph como pujCGAL_triangulate compilaron sin errores. El compilador emite advertencias de APIs deprecadas en C++23 provenientes de CGAL/GMP/Boost (float_denorm_style, denorm_absent), pero estas son externas al código propio y no afectan el comportamiento del programa.

5.4 Aprendizaje

Este obstáculo ilustra una regla importante de C++ con templates: declaración y definición fuera de línea deben ser la misma plantilla, no solo tener el mismo comportamiento en la instanciación. El compilador no hace inferencia entre template<class TData> y template<class TKernel> aunque en la llamada concreta produzcan el mismo tipo. Esta distinción no es evidente cuando se escribe el header sin tener la implementación enfrente, y es el tipo de error que el compilador reporta con mensajes poco intuitivos ('definition does not match any declaration').

Esta sección cierra el ciclo de diseño inicial. Las secciones siguientes documentan la implementación completa.

6. Implementación del grafo dual: DualGraph y build_dual_graph

6.1 La clase DualGraph

Siguiendo el patrón de la biblioteca del profesor —declaración en .h, implementación en .hxx, miembros protegidos con iteradores públicos— creamos DualGraph<TKernel> con cinco estructuras internas: m_Barycenters (vector de TPoint_2 con un baricentro por triángulo), m_PointInfinity (el punto P∞ fuera de la caja envolvente), m_IsBoundary (vector<bool> que marca los triángulos que tocan el borde), m_InternalEdges y m_ExternalEdges (listas de pares de índices). La m_AdjacencyMatrix es una matriz booleana (n+1)×(n+1) donde el índice n corresponde a P∞, lo que permite consultar adyacencia con adjacent(i,j) en O(1) sin casos especiales.

6.2 La función build_dual_graph y el edge_map

La función libre build_dual_graph() opera en seis pasos. Primero inicializa el objeto y copia los baricentros. Segundo, construye el edge_map: un std::map cuya clave es el par canónico {min(a,b), max(a,b)} de índices de vértice y cuyo valor es la lista de triángulos que comparten esa arista. Esta representación canónica elimina la necesidad de buscar duplicados. Tercero, clasifica cada entrada: tamaño 2 = arista interna (conecta dos baricentros), tamaño 1 = arista de frontera (conecta a P∞). Cuarto, llena la matriz de adyacencia. Quinto, calcula P∞ desplazando el centro de la caja envolvente 2× su tamaño hacia la esquina inferior derecha. Sexto, guarda todo en el objeto. El costo es O(n log n), dominado por la inserción en std::map.

6.3 Serialización IO_DualGraph y verificación

Implementamos una especialización de pujCGAL::IO::save() para DualGraph siguiendo el convenio OBJ del proyecto: líneas v para los baricentros con P∞ como último vértice, luego líneas l para las aristas. El visualizador identifica P∞ como el último vértice del archivo sin metadatos adicionales. Ejecutado sobre los diez polígonos de data/, el grado de cada nodo fue siempre 3, confirmando la correctitud.

7. Visualizador C++/VTK: reemplazo de visualizer.py

7.1 Decisión: C++ nativo en lugar de Python/VTK

El visualizador original en Python/VTK se colgaba de forma sistemática. El intérprete pierde respuesta en el event loop de VTK bajo ciertos polígonos y el arranque era lento por el tiempo de importación. Reimplementamos el visualizador como ejecutable C++ nativo (visualizer.cxx), compilado en el mismo CMakeLists.txt. El arranque es inmediato y no hay overhead de intérprete.

7.2 Tres modos: PNGs, GIF e interactivo

El modo por defecto activa SetOffScreenRendering(1) y captura 5 PNGs con vtkWindowToImageFilter + vtkPNGWriter activando una capa adicional por frame. El modo --gif produce esos mismos PNGs y llama a ffmpeg via std::system() con un archivo ffconcat para ensamblarlos en dual_graph.gif. El modo --interactive abre la ventana VTK con toggle de capas (teclas 1, 2, 3), screenshot (s) y salida (q). Cada ejecución crea una carpeta nueva output/result-NNN/ mediante std::filesystem sin sobreescribir resultados anteriores.

8. Estado final y aprendizajes

El pipeline C++ está completo: lectura del polígono, garantía CCW, triangulación CGAL, baricentros, edge_map, grafo dual, P∞, serialización OBJ y visualización VTK. Tres decisiones de diseño resultaron especialmente valiosas: representar P∞ como nodo real con índice n (simplifica toda la lógica); usar std::map con clave canónica {min,max} para el edge_map (la clasificación se reduce a preguntar si el vector tiene tamaño 1 o 2); y separar las herramientas por responsabilidad (polygon_drawer para entrada, dual_graph para el algoritmo, visualizer para la salida). Queda pendiente la triangulación manual por Ear Clipping sin CGAL.

9. Resultados visuales: output del visualizador C++

Las siguientes figuras muestran los 5 PNGs generados por visualizer.cxx en modo por defecto sobre el polígono poly_01.obj (21 vértices). Cada imagen corresponde a un frame capturado con VTK offscreen, activando una capa adicional sobre la anterior. El fondo oscuro, el contorno amarillo, las diagonales azules, los baricentros verdes y las aristas externas en rojo replican la paleta de polygon_drawer.py para coherencia visual entre herramientas.

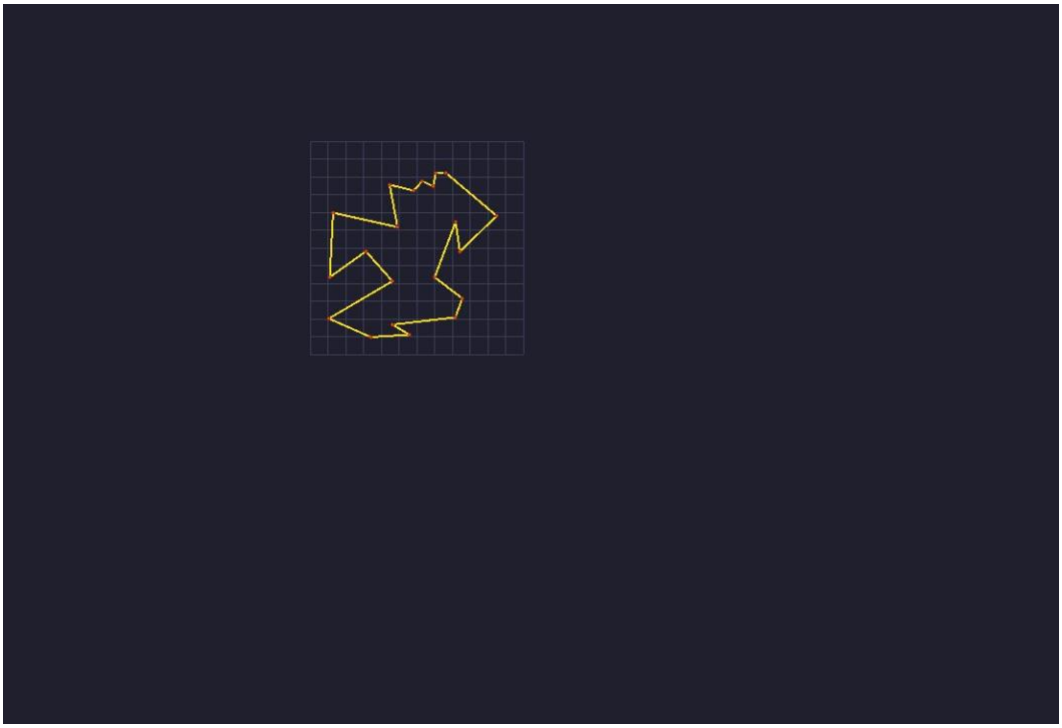

Figura 6. 00_polygon.png — polígono de entrada con 21 vértices (amarillo). Los puntos naranja son los vértices. La grilla de fondo es el mismo fondo que polygon_drawer.py.

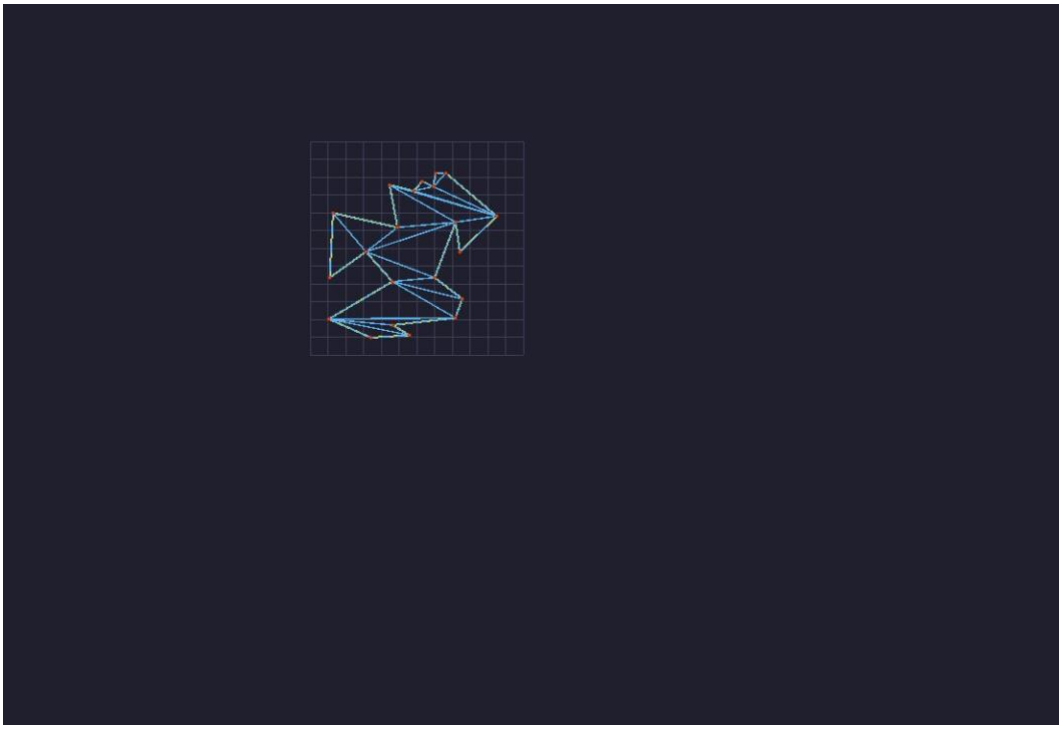

Figura 7. 01_triangulation.png — triangulación CGAL superpuesta (azul claro). Las diagonales dividen el polígono en 19 triángulos.

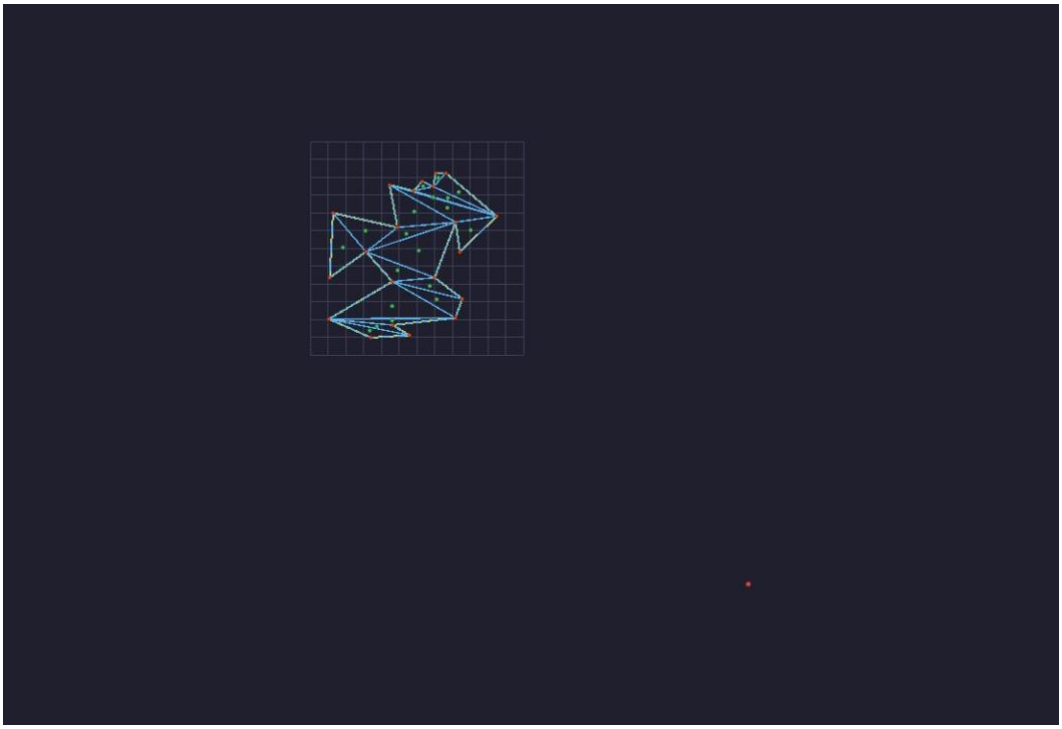

Figura 8. 02_barycenters.png — baricentros de los 19 triángulos (puntos verdes) y P∞ (punto rojo, esquina inferior derecha de la pantalla).

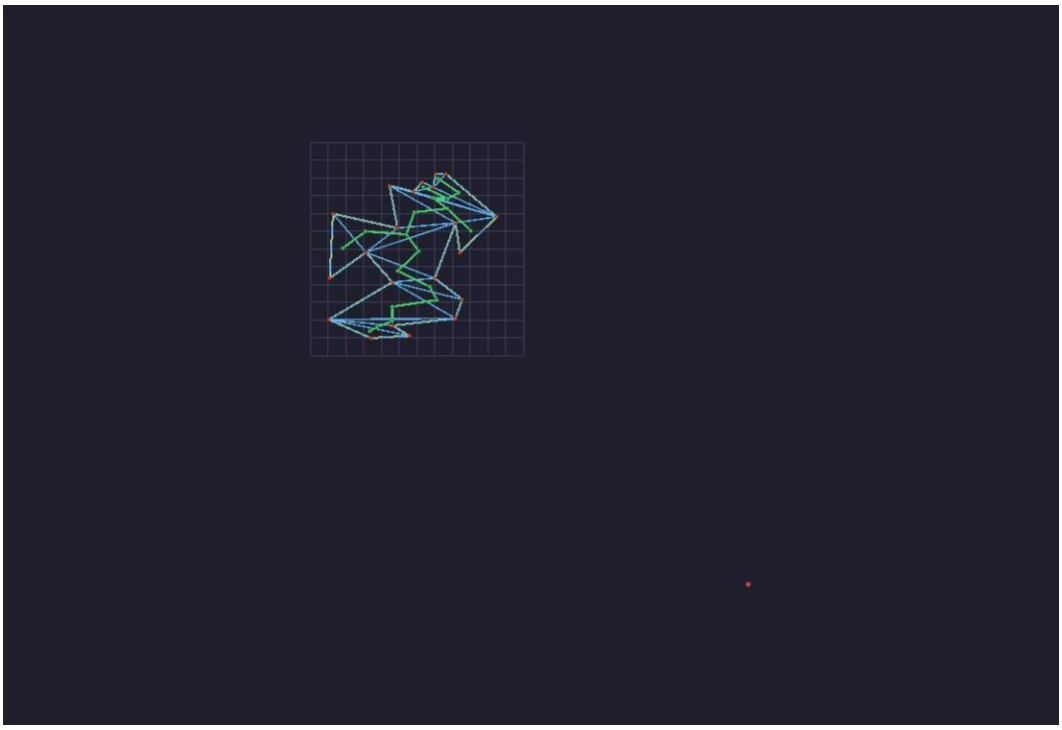

Figura 9. 03_internal_edges.png — aristas internas del grafo dual (verde): conectan baricentros de triángulos que comparten una diagonal.

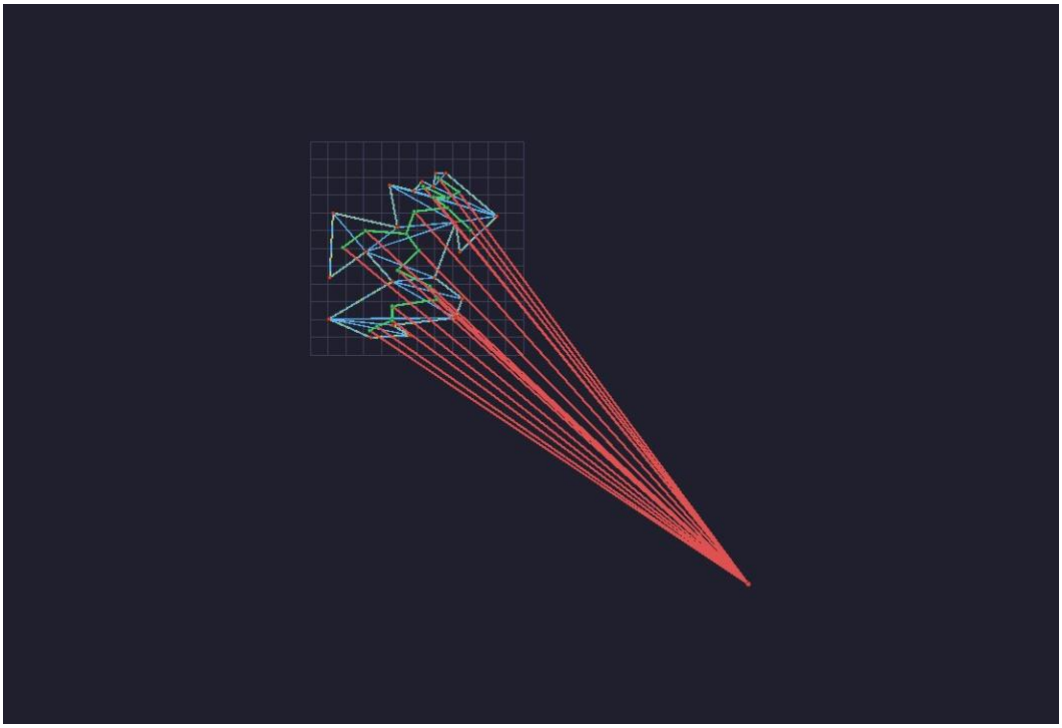

Figura 10. 04_dual_complete.png — grafo dual completo. Las aristas rojas son las externas que conectan cada baricentro de frontera con P∞ (punto rojo, fuera de la caja envolvente). Todo nodo tiene grado 3.